Python-Exception Handling

Objective

OBJECT™ TECHNOLOGIES

- What is Exception
- Common Exceptions
- Exception Handling
- Except clause
- Try finally clause
- Argument of an Exception
- User Defined Exceptions

What is Exception

OBJECT™ TECHNOLOGIES

- An exception can be defined as an **unusual condition in a program resulting in the interruption in the flow of the program.**
- Whenever an exception occurs, the program stops the execution, and thus the further code is not executed. Therefore, an exception is the run-time errors that are unable to handle to Python script.
- An exception is a Python object that represents an error.
- If we do not handle the exception, the interpreter doesn't execute all the code that exists after the exception.

Common Exceptions

OBJECT TECHNOLOGIES

Python provides the number of built-in exceptions, but here we are describing the common standard exceptions.
A list of common exceptions that can be thrown from a standard Python program is given below:

1. **ZeroDivisionError**: Occurs when a number is divided by zero.
2. **NameError**: It occurs when a name is not found. It may be local or global.
3. **IndentationError**: If incorrect indentation is given.
4. **IOError**: It occurs when Input Output operation fails.
5. **EOFError**: It occurs when the end of the file is reached, and yet operations are being performed.
6. **IndexError**: Raised when the index of a sequence is out of range.
7. **KeyError** : Raised when a key is not found in a dictionary.
8. **SyntaxError**: Raised by parser when syntax error is encountered.
9. **TypeError** : Raised when a function or operation is applied to an object of incorrect type.
10. **ValueError** : Raised when a function gets an argument of correct type but improper value.

www.object.co.in

Exception Handling

- **The try-expect statement**
  If the Python program contains suspicious code that may throw the exception, we must place that code in the try block.
- The try block must be followed with the **except statement**, which contains a block of code that will be executed if there is some exception in the try block.

Syntax:

```
try:
   #block of code

except Exception1:
   #block of code

except Exception2:
   #block of code

#other code
```


www.object.co.in

Example 1

```
try:
    a = int(input("Enter a:"))
    b = int(input("Enter b:"))
    c = a/b
except:
    print("Can't divide with zero")
```

Output:

```
Enter a:10
Enter b:0
Can't divide with zero
```

else clause with try catch

- We can also use the **else statement** with the try-except statement in which, we can place the code which will be executed in the scenario if no exception occurs in the try block.

  The syntax of else statement with the try-except statement:

```
try:
    #block of code

except Exception1:
    #block of code

else:
    #this code executes if no except block is executed
```


www.object.co.in

Except clause

- The except statement with no exception.
- Python provides the flexibility not to specify the name of exception with the exception statement.

Example

```
try:
    a = int(input("Enter a:"))
    b = int(input("Enter b:"))
    c = a/b;
    print("a/b = %d"%c)
except:
    print("can't divide by zero")
else:
    print("Hi I am else block")
```


www.object.co.in

Declaring Multiple Exceptions

- The Python allows us to declare the multiple exceptions with the except clause.
- Declaring multiple exceptions is useful in the cases where a try block throws multiple exceptions.

- **Syntax**

```
try:
    #block of code

except (<Exception 1>, <Exception 2>, <Exception 3>,
...<Exception n>)
    #block of code

else:
    #block of code
```


www.object.co.in

Example:

```
try:
   a=10/0;
except(ArithmeticError, IOError):
   print("Arithmetic Exception")
else:
   print("Successfully Done")
```

Output:

Arithmetic Exception

Points to remember

1. Python facilitates us to not specify the exception with the except statement.
2. We can declare multiple exceptions in the except statement since the try block may contain the statements which throw the different type of exceptions.
3. We can also specify an else block along with the try-except statement, which will be executed if no exception is raised in the try block.
4. The statements that don't throw the exception should be placed inside the else block.

Try finally clause

- Python provides the optional finally statement, which is used with the try statement.
- It is executed no matter what exception occurs and used to release the external resource.
- The finally block provides a guarantee of the execution.
- We can use the finally block with the try block in which we can pace the necessary code, which must be executed before the try statement throws an exception.

Syntax

```
try:
    # block of code
    # this may throw an exception
finally:
    # block of code
    # this will always be executed
```


www.object.co.in

Syntax:

```
try:
    # Some Code....

except:
    # optional block
    # Handling of exception (if required)

else:
    # execute if no exception

finally:
    # Some code .....(always executed)
```

```
try:
    k = 5//0 # raises divide by zero exception.
    print(k)

# handles zerodivision exception
except ZeroDivisionError:
    print("Can't divide by zero")

finally:
    # always executed regardless of exception generation.
    print('This is always executed')
```

Output:

```
Can't divide by zero
This is always executed
```

Argument of an Exception

OBJECT™ TECHNOLOGIES

1. It can be used to gain additional information about the error encountered.
2. As contents of an Argument can vary depending upon different types of Exceptions in Python, Variables can be supplied to the Exceptions to capture the essence of the encountered errors.
3. Same error can occur of different causes, Arguments helps us identify the specific cause for an error using the except clause.
4. It can also be used to trap multiple exceptions, by using a variable to follow the tuple of Exceptions.

Syntax:

```
try:
  You do your operations here;
  ......................
except ExceptionType  as Argument:
  You can print value of Argument here...
```


www.object.co.i

- If you write the code to handle a single exception, you can have a variable follow the name of the exception in the except statement.
- If you are trapping multiple exceptions, you can have a variable follow the tuple of the exception.
- This variable receives the value of the exception mostly containing the cause of the exception.
- The variable can receive a single value or multiple values in the form of a tuple. **This tuple usually contains the error string, the error number, and an error location.**

Example:

```
# Define a function here.
def temp_convert(var):
  try:
    return int(var)
  except ValueError as  Argument:
    print ("The argument does not contain numbers ", Argument)

# Call above function here.
temp_convert("xyz");
```

Output:

```
The argument does not contain numbers
invalid literal for int() with base 10: 'xyz'
```


www.object.co.in

```
try:
    a = int(input("Enter a:"))
    b = int(input("Enter b:"))
    c = a/b
    print("a/b = %d"%c)
    # Using exception object with the except statement
except Exception as e:
    print("can't divide by zero")
    print(e)
else:
    print("Hi I am else block")
```

Output:

```
Enter a:10
Enter b:0
can't divide by zero
division by zero
```


www.object.co.in

Raise an exception

- As a Python developer you can choose to throw an exception if a condition occurs.
- To throw (or raise) an exception, use the raise keyword.

Example

Raise an error and stop the program if x is lower than 0:

```
x = -1
if x < 0:
  raise Exception("Sorry, no numbers below zero")
```

- You can define what kind of error to raise, and the text to print to the user.

Example

Raise a TypeError if x is not an integer:

```
x = "hello"
if not type(x) is int:
  raise TypeError("Only integers are allowed")
```


www.object.co.in

User Defined Exceptions

The Python allows us to create our exceptions that can be raised from the program and caught using the except clause.

Example

```
class ErrorInCode(Exception):
    def __init__(self, data):
        self.data = data
    def __str__(self):
        return repr(self.data)
try:
    raise ErrorInCode(2000)
except ErrorInCode as ae:
    print("Received error:", ae.data)
```

Output:

Received error: 2000

www.object.co.in

Example:

```
# define Python user-defined exceptions
class Error(Exception):
    """Base class for other exceptions"""
    pass

class ValueTooSmallError(Error):
    """Raised when the input value is too small"""
    pass

class ValueTooLargeError(Error):
    """Raised when the input value is too large"""
    pass

# you need to guess this number
number = 10
```

```
# user guesses a number until he/she gets it right
while True:
    try:
        i_num = int(input("Enter a number: "))
        if i_num < number:
            raise ValueTooSmallError
        elif i_num > number:
            raise ValueTooLargeError
        break
    except ValueTooSmallError:
        print("This value is too small, try again!")
        print()
    except ValueTooLargeError:
        print("This value is too large, try again!")
        print()

print("Congratulations! You guessed it correctly.")
```

- **Output**:

```
Enter a number: 12
This value is too large, try again!

Enter a number: 0
This value is too small, try again!

Enter a number: 8
This value is too small, try again!

Enter a number: 10
Congratulations! You guessed it correctly.
```

Summary

OBJECT™ TECHNOLOGIES

- What is Exception
- Common Exceptions
- Exception Handling
- Except clause
- Try finally clause
- Argument of an Exception
- User Defined Exceptions

Thank You